

Assignment 2

Zhuo Han (zhuohan3)
zhuo.han@my.cityu.edu.hk

November 30, 2025

Chapter 3

P8

In `rdt3.0` the receiver has the same finite-state machine as in `rdt2.2`, because all of the new complexity (timers and timeout retransmissions) is added only at the sender. The receiver simply has to handle bit errors and duplicate packets while ensuring in-order delivery, as was required when under a `rdt2.2` situation. The FSM is shown in Figure 1.

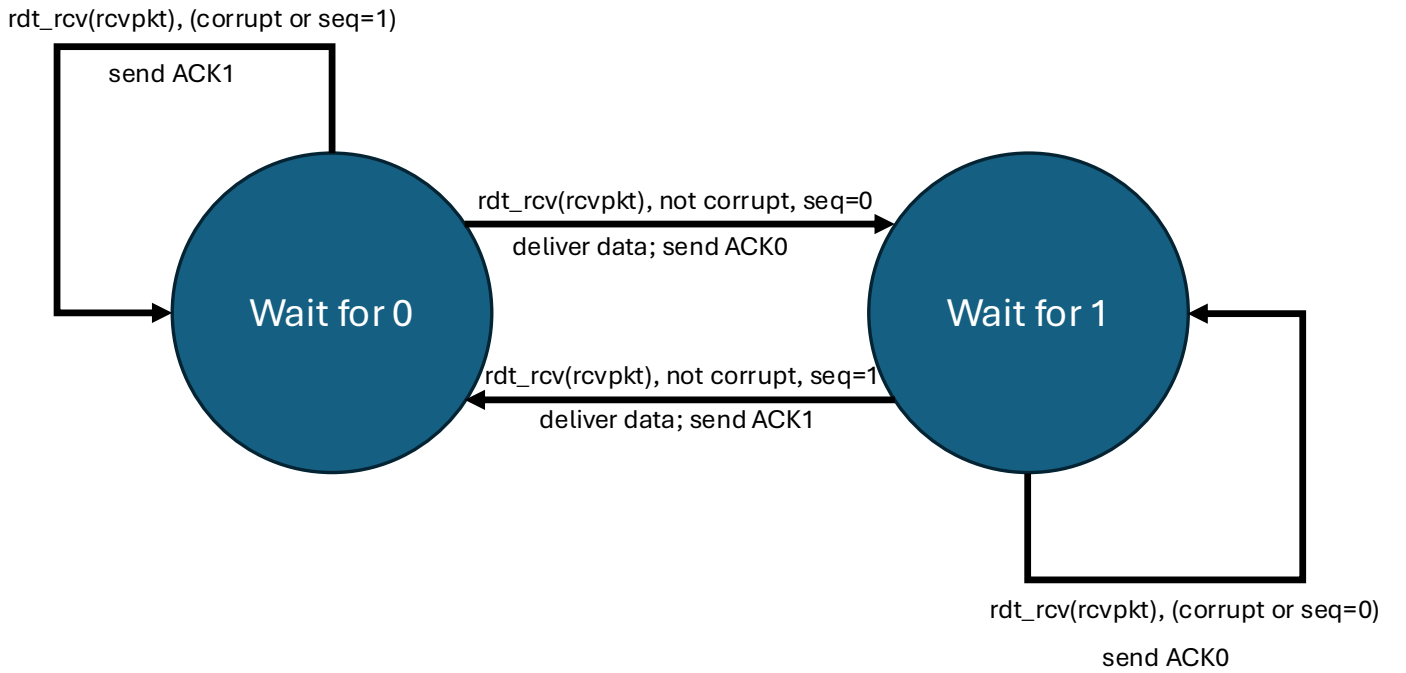


Figure 1: The FSM for the receiver side of protocol `rdt3.0`.

P22

a.

Consider first the extreme case where the sender has already received ACKs for all packets up to $k - 1$. Then every packet with sequence number less than k is acknowledged at the sender, so the leftmost point of the sender's window is at k . And since the window size is 4, the window in this case covers the sequence numbers $\{k, k + 1, k + 2, k + 3\}$. At the other extreme, suppose none of the ACKs for the last N packets before k have yet reached the sender. The receiver has accepted packets $k - N, k - N + 1, \dots, k - 1$, but the sender still considers all of these unacknowledged. Then the oldest unacknowledged packet at the sender is $k - N$, and the window must span the four consecutive sequence numbers from $k - N$ through $k - 1$.

Between these two extremes, the leftmost point of the window can be anywhere from $k - N$ up to k . Thus, at time t the sender's window always has size 4 and its leftmost point lies somewhere in the range $[k - N, k]$. The possible sets of sequence numbers in the window are:

$$\{k - 4, k - 3, k - 2, k - 1\}, \quad (1)$$

$$\{k - 3, k - 2, k - 1, k\}, \quad (2)$$

$$\{k - 2, k - 1, k, k + 1\}, \quad (3)$$

$$\{k - 1, k, k + 1, k + 2\}, \quad (4)$$

$$\{k, k + 1, k + 2, k + 3\} \quad (5)$$

b.

If the receiver is currently waiting for packet k , then it must already have received (and sent ACKs for) packet $k - 1$ and the $N - 1$ packets before it. Suppose that none of these N ACKs has yet arrived at the sender, ACKs with values in the range $[k - N, k - 1]$ may still be propagating back towards the sender. On the other hand, the sender has already transmitted packets with sequence numbers from $k - N$ up through $k - 1$. Because the channel does not reorder messages, by the time the sender has sent packet $k - 1$ the channel does not reorder messages, by the time the sender has sent packet $k - N$. Hence, no ACK with value less than $k - N$ can still be in flight.

Combining these arguments, the ACK field in any ACK message that may currently be on its way back to the sender must be in the interval $[k - N, k - 1]$. For the given window size $N = 4$, the possible ACK values in flight at time t are therefore $k - 4, k - 3, k - 2$, and $k - 1$.

Chapter 4 & 5

P4

a.

Packets whose destination is host H3 are forwarded out link interface 3.

b.

No. The switch's forwarding rule matches only on the destination address, so it cannot distinguish between packets from $H1 \rightarrow H3$ and from $H2 \rightarrow H3$. Any rule that forwards traffic to H3 will treat both sources identically, so it cannot block $H1$ -H3 traffic while still allowing $H2$ -H3 traffic.

Paper Reading

1)

According to the paper, Internet routing wastes performance mainly because it makes decisions with very limited information. Routers usually choose routes using simple or policy based metrics such as AS hop count, instead of using actual measurements of delay, loss or available bandwidth on different paths. This means that traffic may be sent along routes that are longer or more congested even when better alternatives exist. Business relationships between networks further narrow the set of usable paths, since providers tend to hand traffic to other domains as early as possible or avoid carrying transit traffic. Load balancing is mostly done manually by operators and is updated only occasionally, so routing reacts slowly to changes in demand. In addition, standard routing forwards packets for a destination along a single chosen path rather than spreading them across several good paths. Together these design choices lead to unnecessary latency, higher packet loss and underuse of the physical capacity in the network.

2)

For the transport layer, the paper focuses on TCP and argues that its behaviour is often too conservative and too blind to the real network state. TCP starts every connection knowing almost nothing about path characteristics and has to probe cautiously during the handshake and slow start. For short flows this probing phase occupies a large fraction of the transfer, so connection setup delay and slow window growth have a big impact. A large RTT slows down acknowledgements and window increase, and even a moderate packet loss rate triggers strong reductions of the sending rate. Since TCP treats each loss as a sign of congestion, it cannot distinguish between genuine congestion and random loss, and therefore may back off more than necessary. If an early packet such as a SYN is lost, the resulting timeout can be long compared with the size of the flow, which further hurts completion time. Overall, because TCP only observes the path from the edge and reacts cautiously to any hint of trouble, the achieved throughput and latency of many connections are far worse than what the underlying links could in principle support.

3)

The paper suggests that better performance can be obtained if both routing and transport are guided by richer measurement information gathered inside the network. On the routing side, routers or overlay nodes can measure latency, loss and available bandwidth on different paths and use these measurements when selecting routes, instead of relying purely on static metrics and policies. When several good routes exist, traffic can be split among them so that no single path is unnecessarily overloaded and the total capacity is better utilised. Routing policies can also be adapted to the needs of different applications, for example preferring stable low-delay paths for interactive traffic or paths with predictable loss behaviour for streaming. On the transport side, edge routers that see many flows on a path can estimate typical RTTs and fair share bandwidth and provide this information to end hosts. With such hints, hosts can choose more reasonable initial timeouts and initial congestion windows and avoid both overshooting and overly aggressive backoff. In this way, more informed routing combined with more informed transport can substantially improve end-to-end performance without redesigning the basic Internet architecture.